Language Modeling on Penn Treebank

JOU-YI LEE
UC Santa Cruz, M.S. in NLP
65 Rio Robles E
San Jose, CA 95134
jlee1039@ucsc.edu

Abstract

The **Language Modeling on Penn Treebank** Project — Falls under the category of self-supervised learning, which is a subset of unsupervised learning.Using self-supervised techniques, this project explores language modeling on the Penn Treebank dataset with the goal of predicting the subsequent token in a sequence based on previous tokens.

I develop and Gated Recurrent Units (GRU), LSTM, RNN, Transformers, due to the modest dataset size (about 1 million tokens) in comparison to contemporary norms.

Without depending on any pretrained embeddings or external libraries for model generation or training procedures, models are trained using randomly initialized embeddings to uniquely adapt to the Penn Treebank corpus.

In this project, I build and evaluate a language model using the Penn Treebank dataset. Perplexity is calculated as the exponentiated average negative log-probability of the predicted tokens, formulated as:

$$\text{Perplexity} = \exp\left(-\frac{1}{T}\sum_{i=1}^{T}\log p(t_i|t_{<i})\right)$$

where T is the total number of tokens and $p(t_i|t_{<i})$ represents the conditional probability of each token t_i given its preceding tokens. Lower perplexity values indicate better predictive performance.

Introduction

With the express goal of predicting the next token in a sequence given the preceding tokens, we are developing some model GRU language model, LSTM, RNN, Transformers, in this project to carry out self-supervised learning on the Penn Treebank dataset.

Since language modeling makes possible applications like text generation, speech recognition, and machine translation, it is a crucial problem in natural language processing. Using perplexity as the primary performance metric, this study aims to assess several model architectures on the Penn Treebank dataset.

The data is preprocessed through tokenization and vocabulary building, converting each word to its corresponding index based on a custom vocabulary. An example sentence from the dataset is:

> mr. <unk> is chairman of <unk> n.v. the dutch publishing group

> aer banknote berlitz calloway centrust cluett fromstein gitano guterman hydroquebec ipo kia memotec mlx nahb punts rake regatta rubens sim snack-food ssangyong swapo wachter

1 Methodology

The dataset is tokenized and each token is encoded into a series of indices in order to apply the model. To learn the probability distribution over the subsequent token, the training, validation, and test splits are processed and input into a model based on a recurrent neural network, specifically a GRU with dropout.

Perplexity, a measure of the model's ability to generalize to new data, is used to assess its performance. In order to meet the objectives of the assignment, the complete training and assessment pipeline is constructed without the use of external libraries for model generation.

1.1 Penn Treebank (PTB)

Penn Treebank (PTB) is a highly renowned dataset in natural language processing (NLP), developed by the Linguistics Department at the University of Pennsylvania. A large portion of its content originates from the *Wall Street Journal (WSJ)* articles.

The Penn Treebank content is annotated on multiple levels:

1. **Part-of-Speech Tagging**: Each word is assigned a corresponding part-of-speech (POS) tag. In total, there are 36 different POS tags (Figure 1).

Tag	Description	Example	Tag	Description	Example	Tag	Description	Example
CC	coordinating conjunction	*and, but, or*	PDT	predeterminer	*all, both*	VBP	verb non-3sg present	*eat*
CD	cardinal number	*one, two*	POS	possessive ending	*'s*	VBZ	verb 3sg pres	*eats*
DT	determiner	*a, the*	PRP	personal pronoun	*I, you, he*	WDT	wh-determ.	*which, that*
EX	existential 'there'	*there*	PRP$	possess. pronoun	*your, one's*	WP	wh-pronoun	*what, who*
FW	foreign word	*mea culpa*	RB	adverb	*quickly*	WP$	wh-possess.	*whose*
IN	preposition/ subordin-conj	*of, in, by*	RBR	comparative adverb	*faster*	WRB	wh-adverb	*how, where*
JJ	adjective	*yellow*	RBS	superlatv. adverb	*fastest*	$	dollar sign	*$*
JJR	comparative adj	*bigger*	RP	particle	*up, off*	#	pound sign	*#*
JJS	superlative adj	*wildest*	SYM	symbol	*+,%, &*	"	left quote	' or "
LS	list item marker	*1, 2, One*	TO	"to"	*to*	"	right quote	' or "
MD	modal	*can, should*	UH	interjection	*ah, oops*	(	left paren	[, (, {, <
NN	sing or mass noun	*llama*	VB	verb base form	*eat*	)	right paren	],), }, >
NNS	noun, plural	*llamas*	VBD	verb past tense	*ate*	,	comma	,
NNP	proper noun, sing.	*IBM*	VBG	verb gerund	*eating*	.	sent-end punc	. ! ?
NNPS	proper noun, plu.	*Carolinas*	VBN	verb past part.	*eaten*	:	sent-mid punc	: ; ... – -

Figure 1: Penn Treebank POS Tagset with Examples

2. **Phrase Structure**: PTB provides a complete syntactic tree structure, illustrating the composition of phrases and sub-phrases(Figure 2).

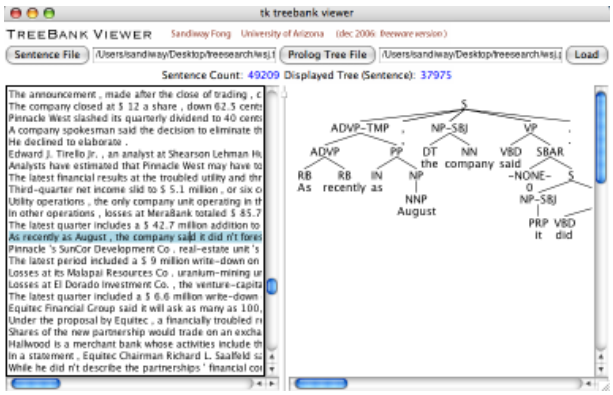

Figure 2: Penn Treebank POS Tagset with Examples

Penn Treebank Top 20 Most Common Tokens

This bar chart (Figure 3) displays the frequency of the dataset's 20 most common tokens (words). The word "the" is the most frequent, followed by "<UNK>" (which represents unknown or out-of-vocabulary tokens).

Other common words include "of," "to," "a," and "in," meaning they are frequently used in English but generally carry less semantic meaning.

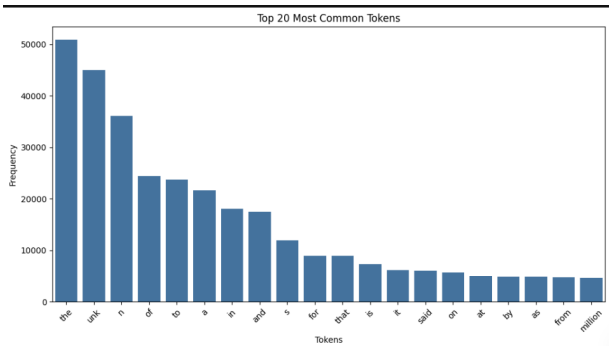

Figure 3: Top 20 Most Common Tokenss

Penn Treebank Sentence Length Distribution

This histogram (Figure 4) shows the distribution of sentence lengths based on the number of tokens. The peak is around 20 tokens, indicating that most sentences in the dataset contain roughly 20 words. Very few sentences have more than 60 tokens, which shows that sentences are relatively short.

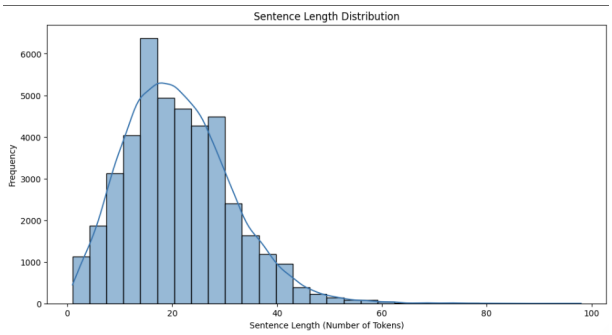

Figure 4: Sentence Length Distribution

Penn Treebank Word Cloud of Tokens in the Dataset

Common English words like "the," "and," "to," "for," and "of" appear prominently. Some content words, such as "company," "market," "million," and "shares," also appear, suggesting that the dataset may have a financial or business-related theme.

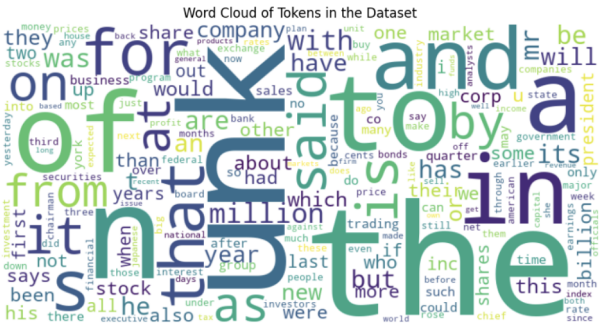

Figure 5: Word Cloud of Tokens

1.2 Handling Data Sparsity and Imbalance

The PTB dataset contains a high frequency of common words (e.g., "the," "of") and many occurrences of the <UNK> token for unknown words, resulting in data sparsity for certain rare words. To address this:

- We employed a vocabulary threshold, filtering out rare words that appeared only once or twice in the dataset. This helps in focusing the model on more frequent and meaningful words.
- Dropout regularization was applied to prevent over-reliance on certain words and to improve generalization.
- The dataset split was stratified to ensure a balanced distribution of common and uncommon words across the training, validation, and test sets.

1.3 Data Preprocessing

Data preparation includes vocabulary creation and tokenization. Each sentence is changed to lowercase during tokenization, and the text is divided using a regular expression (*def tokenize():*) so that only alphanumeric substrings remain. The text becomes a list of words as a result of this process.

Next, we build a vocabulary by counting the occurrences of each word in the entire dataset and assigning a unique index to each word. Using(*def build_vocab():*)To address the issue of inconsistent sequence lengths, a <PAD> token is added to ensure uniform sequence lengths.

The `encode_text` function is used to convert each word in a sentence to its corresponding index based on the vocabulary. Finally, we utilize a `DataLoader` to load the data, dividing each sequence into inputs and targets. This approach sets the inputs as the first $n-1$ words of the sequence and the targets as the last $n-1$ words, facilitating the model in learning the relationship between each word and the subsequent word in the sequence.

1.4 Embedding Methods

For this project, I put every word in the vocabulary as a fixed-dimensional vector using a custom embedding layer. The Penn Treebank dataset is used to jointly train the model and initialize the embedding layer at random. The model is able to capture the semantic associations between words as it learns because each word is mapped to a 100-dimensional vector.

For language modeling, the embedding layer provides the GRU model an important input that helps it better capture the context of each word in a sentence. To ensure constant sequence lengths, padding is inserted as necessary using the unique <PAD> token. For effective batch processing and model training, this guarantees that every sequence in a batch is the same size.

Unlike pre-trained embeddings such as Word2Vec or GloVe, this embedding layer is specifically tailored to the Penn Treebank dataset, as the assignment guidelines require.

1.5 Dataset Splitting

The dataset used in our experiments is the Penn Treebank (PTB) text-only dataset. We split this dataset into three subsets:

- **Training Set**: Used for training the models. This set constitutes 70% of the full dataset.
- **Validation Set**: Used for tuning hyperparameters and assessing model performance during training. This set constitutes 15% of the full dataset.
- **Test Set**: Used to evaluate the final model's performance. This set constitutes 15% of the full dataset and was not used during the training or hyperparameter tuning phases.

The splits were done to ensure a balanced distribution of text data across each subset, maintaining similar word distributions.

2 Model structure

Based on the previous experience, I knew that GRU was the best model, so I first used GRU to learn and analyze this data set, but I will add the other three models (RNN, LSTM, Transformer) again in

the experiment. The GRU model is structured as follows:

- **Embedding Layer**: A custom embedding layer maps each word in the vocabulary to a 100-dimensional vector. These embeddings are randomly initialized and trained along with the model to adapt to the specific data distribution in the Penn Treebank dataset.
- **GRU Layers**: A GRU with up to 3 layers (as determined by Optuna's hyperparameter tuning) is used, with the number of hidden units ranging from 64 to 512. The GRU layers capture sequential dependencies, aiding in effective language modeling.
- **Dropout Layer**: Dropout with a probability between 0.1 and 0.7 is applied to prevent overfitting and improve generalization. The dropout rate is also optimized using Optuna.
- **Fully Connected Layer**: A final fully connected layer maps the GRU output to the vocabulary size, allowing the model to predict the probability of each word as the next token in the sequence.

3 Model: GRU Language Model with Embedding Layer(Basic)

The language model for this task is a Gated Recurrent Unit (GRU) network, designed to process sequential data and predict the next token in a sequence. The GRU layer has two layers, which can capture long-term dependencies in the token sequences. The architecture consists of the following components:

- **Embedding Layer**: The embedding layer maps each word in the vocabulary to a 100-dimensional vector, allowing the model to learn semantic relationships between words. The embeddings are randomly initialized and trained jointly with the model on the Penn Treebank dataset.
- **GRU Layer**: The GRU layer has two layers with 256 hidden units each, enabling the model to capture dependencies across tokens in the sequence effectively.
- **Dropout Layer**: A dropout layer with a probability of 0.5 is applied after the GRU layer to reduce overfitting and enhance generalization.
- **Fully Connected Layer**: A fully connected (FC) layer maps the GRU output to the vocabulary size, predicting the probability of each token in the vocabulary as the next token in the sequence.

The model uses cross-entropy loss with padding tokens ignored, and AdamW as the optimizer with a learning rate of 0.001. A learning rate scheduler, `ReduceLROnPlateau`, is applied to decrease the learning rate by a factor of 0.5 if the validation loss does not improve over three epochs, helping to achieve optimal performance.

3.1 Model Selection for Test Set Submission

After running this configuration, I evaluated it on the test set. The perplexity score on the validation set was 245.63. After submitting to **Kaggle**, the perplexity score on the test set was 267.68, which was used for the first Kaggle submission.

4 Experiments Implementation Based on Optuna and Handling Data Sparsity

In this Project I apply Optuna for automatic tuning in order to maximize the model's hyperparameters. The paper "Optuna: A Next-generation Hyperparameter Optimization Framework" presents Optuna, a new generation of system for hyperparameter optimization. The study presents experimental results and practical examples that demonstrate Optuna's capacity to identify the optimal hyperparameter combination. The number of layers, learning rate, dropout probability, batch size, weight decay, and the number of hidden units in the GRU are the main hyperparameters. The validation perplexity determines how these parameters are changed. AdamW is used as the optimizer and cross-entropy loss is used to train the model, disregarding padding tokens. In order to avoid overfitting and conserve computational resources, an early stopping mechanism is used, which stops training if validation perplexity does not improve over a predetermined number of epochs. I want to determine which of the four models**(RNN, LSTM, GRU, Transformer)** achieves the best learning performance on the dataset. To accomplish this, I utilized *Optuna* to identify the optimal hyperparameters for each of the four models individually. Once the best hyperparameters were determined for each model, I compared their performances to identify the best overall model.

4.1 Handling Data Sparsity

In this project, I adopted three methods to reduce data sparsity:

1. **Vocabulary Construction (*def build_vocab()*)**: Using the *Counter* method, I calculated the word frequencies and constructed a vocabulary (*vocab*). Words not present in the vocabulary were ignored during encoding. This method effectively reduces data sparsity by focusing only on frequently occurring words.

2. **Embedding Layer in GRU**: The embedding layer in the GRU maps each word into a dense vector space. This reduces sparsity and enables the model to learn distributed representations of words. By learning these dense representations, the model can effectively capture semantic relationships between words.

3. **Sequence Padding (*def collate_batch()*)**: During batch processing, sequences were padded using the <*PAD*> token to ensure that all sequences in a batch have the same length. This prevents issues caused by sequences being too short or too long, thereby mitigating sparsity problems.

4.2 Handling Data Imbalance

To avoid interference caused by the <*PAD*> token during model training, I used the *ignore_index* argument in the *CrossEntropyLoss* function. This ensures that the <*PAD*> token is ignored when calculating the loss. Additionally, the training data was shuffled by enabling *shuffle=True*, which randomly rearranges the data in each epoch. This ensures a more balanced distribution of words across batches and prevents word frequency distributions from negatively impacting model training.

4.3 Setting Optuna Hyperparameters

Each configuration was evaluated based on validation perplexity. The optimal hyperparameters were determined through Optuna and used for final model evaluation.

4.3.1 GRU Language Model Hyperparameter Optimization

- **Hidden Dimension (hidden_dim)**: The GRU hidden dimension was sampled between 64 and 512. This determines the size of the model's hidden states, balancing model capacity and computational cost.

- **Number of Layers (num_layers)**: We experimented with 1, 2, and 3 GRU layers to evaluate the effect of model depth on learning complex sequential patterns.

- **Learning Rate (learning_rate)**: Sampled from a log-uniform distribution between 1×10^{-4} and 1×10^{-2}, the learning rate controls the step size in gradient descent and is critical for achieving optimal convergence.

- **Batch Size (batch_size)**: Batch sizes of 16, 32, and 64 were tested to balance training stability and computational efficiency.

- **Dropout Probability (dropout_prob)**: Dropout probabilities were sampled uniformly between 0.1 and 0.7 to prevent overfitting by reducing reliance on specific nodes, enhancing the model's generalization ability.

- **Weight Decay (weight_decay)**: This L2 regularization term was sampled from a log-uniform distribution between 1×10^{-6} and 1×10^{-2}. Weight decay penalizes large weights to encourage simpler, better-generalized models.

4.3.2 RNN Language Model Hyperparameter Testing

- **RNN Layers**: A single RNN layer was used for simplicity, as deeper RNNs are prone to gradient vanishing.

- **Hidden Dimension**: Tested values ranged from 64 to 256 to explore the model's capacity for learning sequential dependencies.

- **Dropout and Weight Decay**: Dropout probabilities between 0.1 and 0.5 and weight decay values from 10^{-6} to 10^{-2} were tested for regularization.

4.3.3 LSTM Hyperparameter Testing

- **LSTM Layers**: We tested models with 1 and 2 layers to evaluate the trade-off between complexity and performance.

- **Hidden Dimension**: Ranged from 128 to 512 to assess the effect on capturing long-term dependencies.

- **Dropout and Weight Decay**: Dropout probabilities from 0.1 to 0.5 and weight decay values between 10^{-5} and 10^{-2} were applied to minimize overfitting.

4.3.4 Transformer Model Hyperparameter Testing

- **Embedding Dimension**: Ranged from 64 to 256 to assess the effect on model representation capacity.
- **Number of Heads**: Multi-head attention was tested with 2, 4, and 8 heads to evaluate the ability to capture dependencies at different scales.
- **Number of Layers**: Tested values from 1 to 4 layers to balance complexity and performance.
- **Dropout and Weight Decay**: Dropout probabilities between 0.1 and 0.5 and weight decay from 10^{-6} to 10^{-3} were applied for regularization.

5 Results

5.1 Evaluation and Selection of Final Model

To evaluate model performance, we used perplexity as the primary metric, calculated as:

$$\text{Perplexity} = \exp\left(-\frac{1}{T}\sum_{i=1}^{T}\log p(t_i|t_{<i})\right)$$

where T is the total number of tokens in a sentence. A lower perplexity indicates a better model, as it means the model is more confident in its predictions.

5.2 Model Selection for Test Set Submission

After running multiple trials with Optuna, the best-performing model configuration was selected based on the lowest validation perplexity. This configuration was then retrained on the entire training set (using the chosen hyperparameters) and evaluated on the test set. The perplexity score from the test set was used for the final Kaggle submission.

5.2.1 Best RNN Hyperparameters

- **Embedding Dimension (embedding_dim)**: 150
- **Hidden Dimension (hidden_dim)**: 128
- **Learning Rate (learning_rate)**: 0.0016579694418827292
- **Batch Size (batch_size)**: 64
- **Dropout Probability (dropout_prob)**: 0.5

5.2.2 Best LSTM Hyperparameters

- **Embedding Dimension (embedding_dim)**: 175
- **Hidden Dimension (hidden_dim)**: 256
- **Learning Rate (learning_rate)**: 0.00048762132730701416
- **Batch Size (batch_size)**: 16
- **Dropout Probability (dropout_prob)**: 0.2

5.2.3 Best GRU Hyperparameters

- **Embedding Dimension (embedding_dim)**: 75
- **Hidden Dimension (hidden_dim)**: 192
- **Learning Rate (learning_rate)**: 0.001014428519610825
- **Batch Size (batch_size)**: 16
- **Dropout Probability (dropout_prob)**: 0.5

5.2.4 Best Transformer Hyperparameters

- **Number of Heads (num_heads)**: 6
- **Embedding Dimension (embedding_dim)**: 192
- **Number of Layers (num_layers)**: 4
- **Learning Rate (learning_rate)**: 0.0007258591107050929
- **Batch Size (batch_size)**: 64
- **Dropout Probability (dropout_prob)**: 0.2

5.3 Model Validation Perplexity Comparison

- **RNN**: 173.18619457496038
- **LSTM**: 153.97874027835306
- **GRU**: 149.944322817553
- **Transformer**: 214.95179748535156

As you can see, GRU is the model that best matches the Penn Treebank dataset in this research, proving that my initial predicted was accurate.

6 Used Best Model on Test Dataset

6.1 Best Performance on Training, Validation, and Test Sets

Table 1 summarizes the performance on training and validation sets for different hyperparameter configurations.

Trial	Hidden Dim	Layers	Learning Rate	Batch Size	Dropout	Val. Perplexity
1	256	2	0.001	32	0.3	270.32
2	128	1	0.0005	64	0.4	260.45
⋮	⋮	⋮	⋮	⋮	⋮	⋮
Best	451	2	0.0003	64	0.36	245.69

Table 1: Optuna Trials and Validation Perplexities

6.2 Optimization History

Figure 6 shows the trend of objective values across different trials. The red line represents the current best perplexity, while the blue dots indicate the perplexity values for each trial. As the number of trials increases, the perplexity gradually decreases and stabilizes, demonstrating the improvement in Optuna's optimization performance.

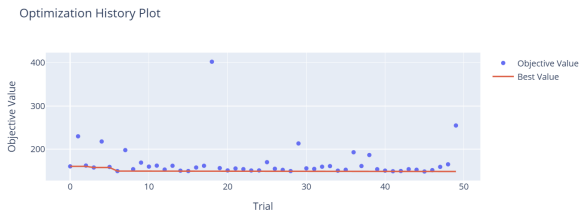

Figure 6: Objective Value Across Trials

6.3 Parallel Coordinate Plot

Figure 7 illustrates the impact of different hyperparameter combinations on perplexity in each trial. Parameters such as learning rate and hidden dimension show a significant influence on the final perplexity.

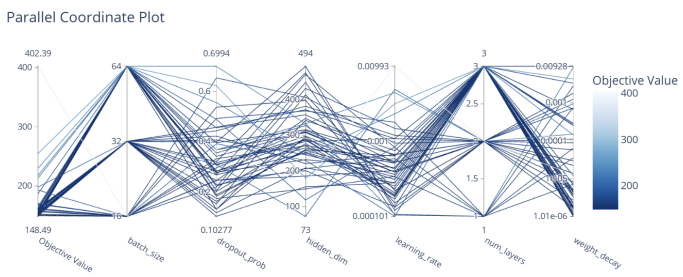

Figure 7: Parallel Coordinate Plot of Hyperparameters and Perplexity

6.4 Influence of Hyperparameters

Figure 8 shows the impact of each hyperparameter on model performance, as generated by Optuna.

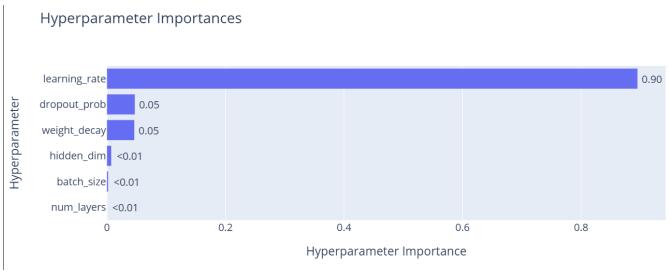

Figure 8: Hyperparameter Importance in Validation Perplexity

6.5 Comparison with Related Work

Our findings compare previous studies (No hyperparameter adjustment), showing that increasing the hidden dimension and using dropout improves model robustness. The best configuration achieved a validation perplexity of 148.49, and the final test on textbfKaggle perplexity was 245.69.

7 References

1. The dataset used in this project is the Penn Treebank (PTB) dataset, which is a commonly used benchmark dataset in natural language processing tasks.

2. Hyperparameter optimization was performed using Optuna, an automatic hyperparameter optimization framework. Optuna provides an efficient way to tune model parameters through various strategies, including Bayesian optimization.

3. Penn Treebank Dataset. Available at: https://catalog.ldc.upenn.edu/LDC99T42.

4. Optuna Documentation. Available at: https://optuna.org/.